

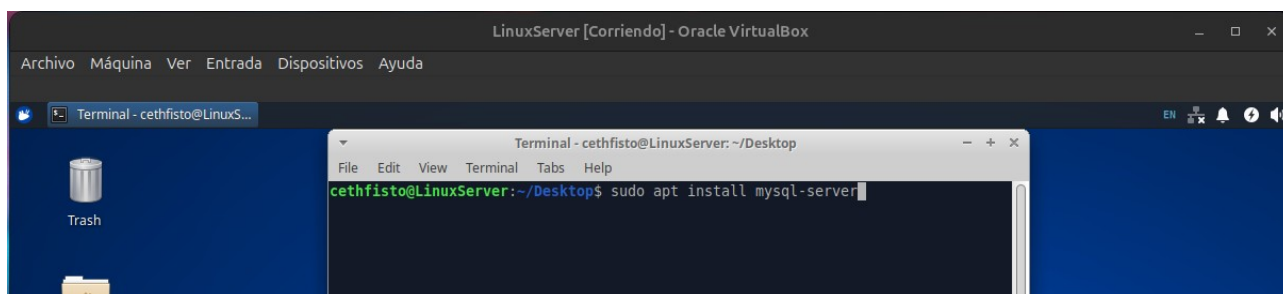
Reto: Bloque 3 Día 11

Fase: 1

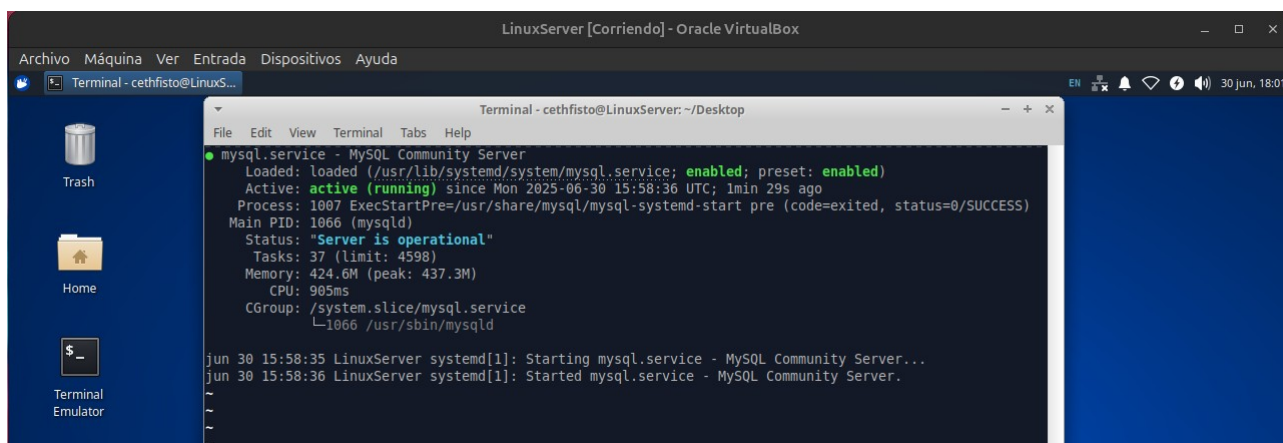
En esta primera parte usaremos MySQL como motor para nuestro servidor de BBDD. Instalaremos MySQL en nuestro servidor Linux, verificaremos el servicio y configuraremos el servicio para que se inicie solo al encender la VM del servidor.

Antes de empezar, añadir que el reto lo he realizado con una VM con Ubuntu Server con una GUI instalada (XFCE), ya que en caso de que deba usar una interfaz gráfica he optado por instalarla previamente.

Para instalar MySQL usaremos el comando `sudo apt install mysql-server`. Hemos elegido MySQL ya que es un motor muy usado en entornos web y de aprendizaje.

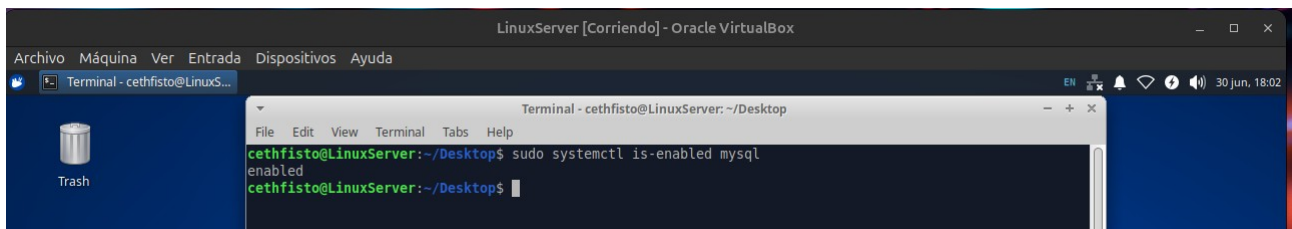


Una vez instalado MySQL, tendremos que verificar el servicio con el comando `sudo systemctl status mysql`.



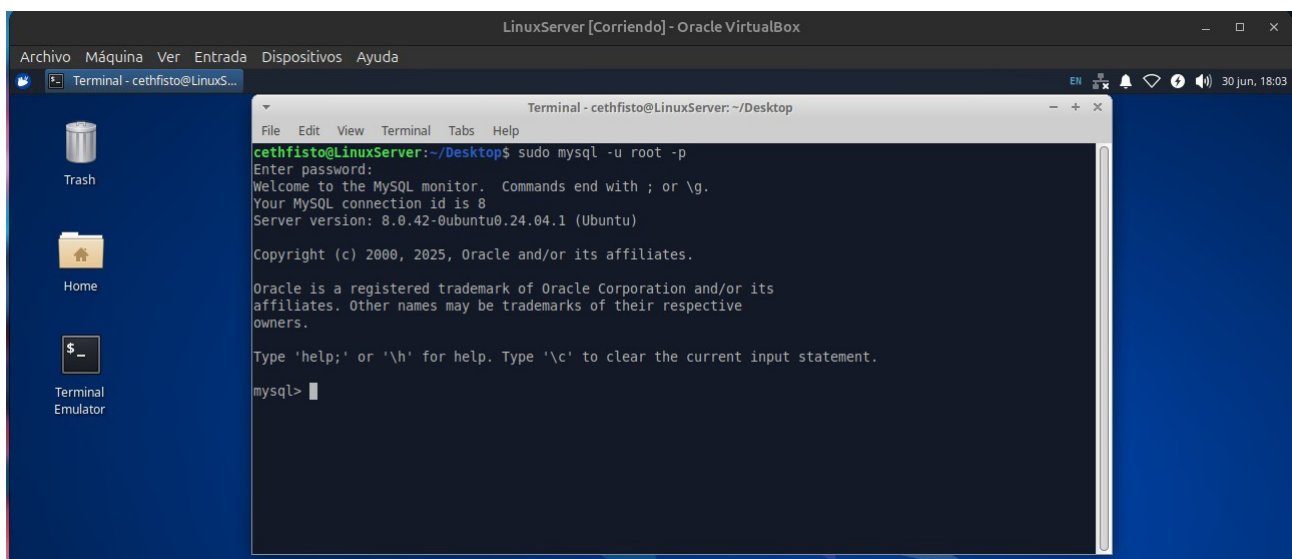
* En este caso podemos ver que está activo y corriendo. Pero en el caso de que no lo tengamos activo, activaremos el servicio con el comando `sudo systemctl start mysql`.

Ya tenemos MySQL instalado y activado. Ahora tenemos que configurarlo para que inicie automáticamente. Podemos comprobar si el servicio está configurado para que se inicie al encender el equipo con el comando `sudo systemctl is-enabled mysql`.



*En el caso de que el resultado fuera disable, usaremos el comando `sudo systemctl enable mysql` para activar el servicio.

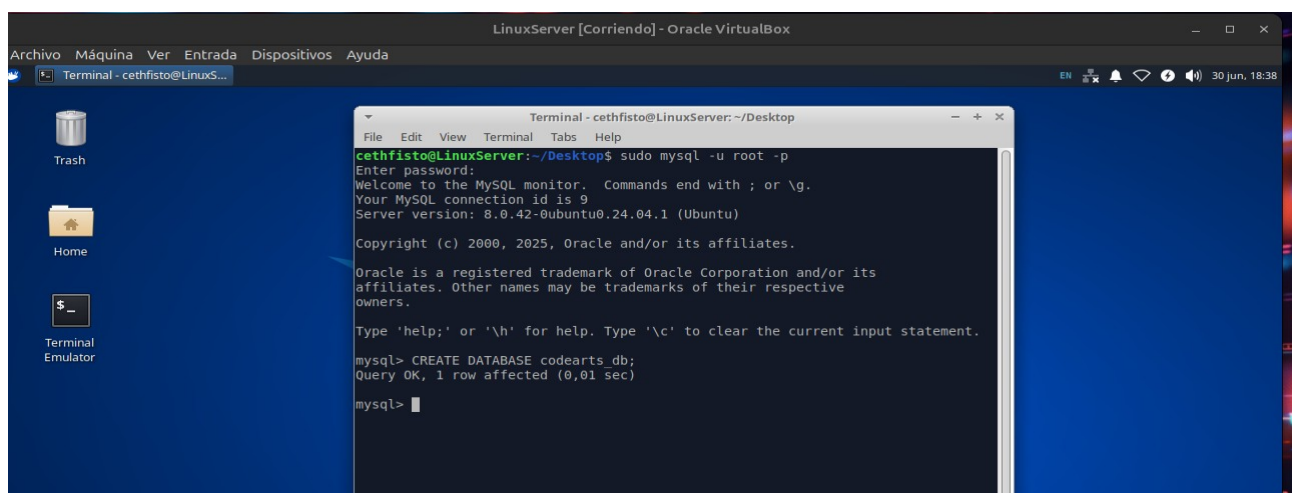
Como prueba final podemos conectarnos a MySQL con el comando `sudo mysql -u root -p` y verificar que todo funciona.



Fase: 2

En esta fase crearemos una BD llamada `codearts_db`. Luego crearemos tres usuarios `admin_db` (con acceso total a la BD), `dev_user` (solo con permiso de W) e `intern_user` (solo con permiso de R). Y asignaremos los permisos anteriormente mencionados.

Iniciaremos sesión en MySQL con nuestro usuario y contraseña root usando el comando `sudo mysql -u root -p`. Y usaremos el comando `CREATE DATABASE [nombreBD];` para crear la BD.



Ahora crearemos los usuarios con sus correspondientes permisos usaremos el comando CREATE USER para crear los usuarios, el comando GRANT para asignar los permisos y el comando FLUSH PRIVILEGES para aplicar los permisos asignados.

```
mysql> CREATE USER 'admin_db'@'localhost' IDENTIFIED BY '1234'
-> ;
Query OK, 0 rows affected (0,01 sec)

mysql> GRANT ALL PRIVILEGES ON codearts_db.* to 'admin_db'@'localhost';
Query OK, 0 rows affected (0,01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,01 sec)

mysql> █
```

* GRANT ALL PRIVILEGES ON codearts_db.* TO 'admin_db'@'localhost';

```
mysql> CREATE USER 'dev_user'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0,01 sec)

Query OK, 0 rows affected (0,01 sec) DELETE ON codearts_db.* TO 'dev_user'@'localhost';

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,00 sec)

mysql> █
```

* GRANT SELECT, INSERT, UPDATE, DELETE ON codearts_db.* TO 'dev_user'@'localhost';

```
mysql> CREATE USER 'intern_user'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0,01 sec)

mysql> GRANT SELECT ON codearts_db.* TO 'intern_user'@'localhost';
Query OK, 0 rows affected (0,01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,00 sec)

mysql> █
```

* GRANT SELECT, INSERT, UPDATE, DELETE ON codearts_db.* TO 'dev_user'@'localhost';

** Como podemos observar hemos usado una contraseña básica para que sea fácil de recordar en caso de que tengamos que hacer un test, lo suyo sería utilizar una contraseña más segura.

A continuación, si queremos ver si hemos aplicado correctamente los permisos con GRANT usaremos el comando SHOW GRANTS. Por ejemplo: SHOW GRANTS FOR 'admin_db'@'localhost';

```
mysql> SHOW GRANTS FOR 'admin_db'@'localhost';
+-----+
| Grants for admin_db@localhost |
+-----+
| GRANT USAGE ON *.* TO `admin_db`@`localhost` |
| GRANT ALL PRIVILEGES ON `codearts_db`.* TO `admin_db`@`localhost` |
+-----+
2 rows in set (0,00 sec)

mysql> █
```

```
mysql> SHOW GRANTS FOR 'dev_user'@'localhost';
+-----+
| Grants for dev_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `dev_user`@`localhost` |
| GRANT SELECT, INSERT, UPDATE, DELETE ON `codearts_db`.* TO `dev_user`@`localhost` |
+-----+
2 rows in set (0,00 sec)

mysql> █
```

```
mysql> SHOW GRANTS FOR 'intern_user'@'localhost';
+-----+
| Grants for intern_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `intern_user`@`localhost` |
| GRANT SELECT ON `codearts_db`.* TO `intern_user`@`localhost` |
+-----+
2 rows in set (0,00 sec)

mysql> █
```

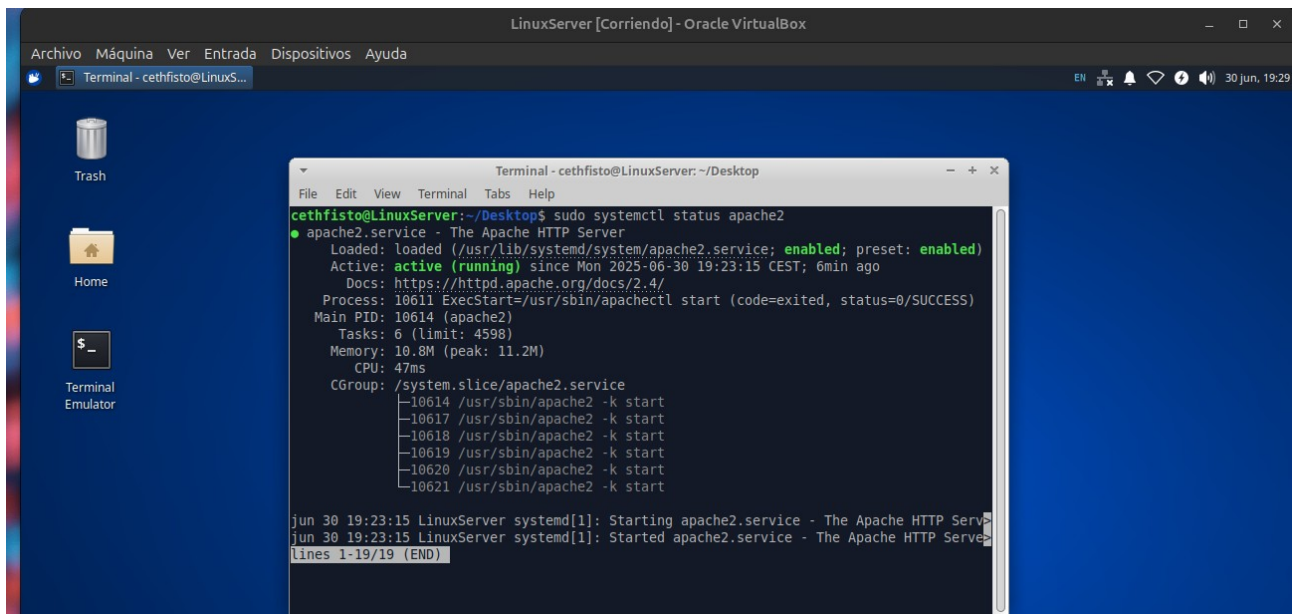
* Podemos ver en las capturas que admin_db tiene todos los permisos, dev_user tiene los permisos de lectura y escritura, y el usuario intern_user solo tiene permisos de lectura.

Fase: 3

En la fase, instalaremos un servidor web (Apache y PHP). Crearemos un script para leer y escribir datos en la BD. Y realizaremos pruebas usando consultas con los comandos, SELECT, INSERT, UPDATE y DELETE desde la aplicación web.

Primero instalaremos los servicios de Apache y PHP con `sudo apt install apache2` y `sudo php libapache2-mod-php php-mysql`.

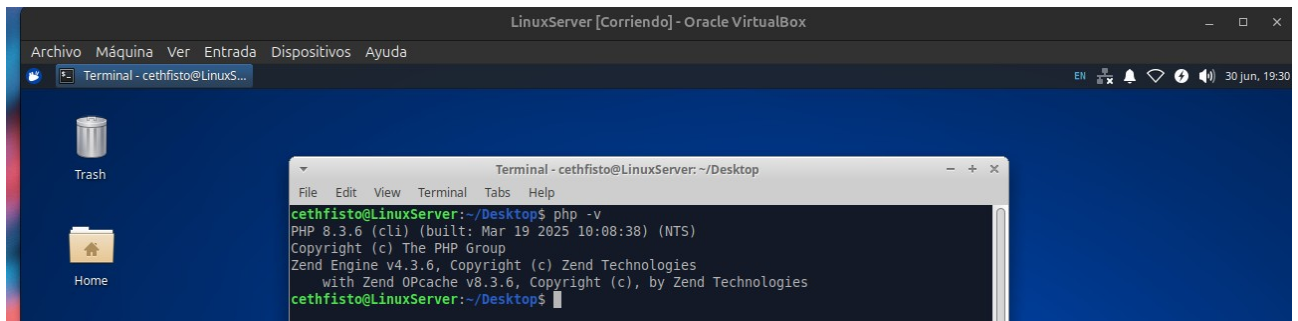
Comprobaremos si los servicios están instalados y funcionando con `sudo systemctl status apache2` y `php -v`.



The screenshot shows a terminal window titled "Terminal - cethfisto@LinuxServer: ~/Desktop". The user has run the command `sudo systemctl status apache2`. The output shows that the `apache2.service` is loaded and enabled, and is currently active (running) since Mon 2025-06-30 19:23:15 CEST. The process is `10611 ExecStart=/usr/sbin/apachectl start` with a status of `0/SUCCESS`. The main PID is `10614 (apache2)`. The tasks are `6 (limit: 4598)`, memory is `10.8M (peak: 11.2M)`, and CPU is `47ms`. The CGroup is `/system.slice/apache2.service`. The output also shows the start of the service and the end of the command.

```
cethfisto@LinuxServer:~/Desktop$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-06-30 19:23:15 CEST; 6min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 10611 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 10614 (apache2)
       Tasks: 6 (limit: 4598)
      Memory: 10.8M (peak: 11.2M)
         CPU: 47ms
       CGroup: /system.slice/apache2.service
              └─10614 /usr/sbin/apache2 -k start
                └─10617 /usr/sbin/apache2 -k start
                  └─10618 /usr/sbin/apache2 -k start
                    └─10619 /usr/sbin/apache2 -k start
                      └─10620 /usr/sbin/apache2 -k start
                        └─10621 /usr/sbin/apache2 -k start

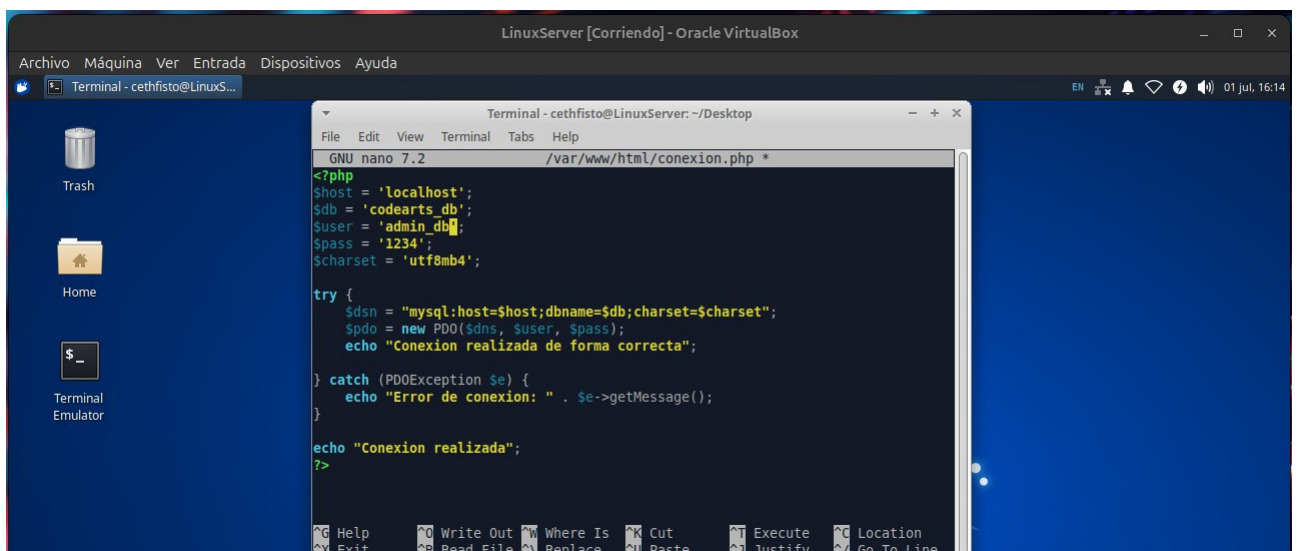
jun 30 19:23:15 LinuxServer systemd[1]: Starting apache2.service - The Apache HTTP Serv>
jun 30 19:23:15 LinuxServer systemd[1]: Started apache2.service - The Apache HTTP Serve>
lines 1-19/19 (END)
```



The screenshot shows a terminal window titled "Terminal - cethfisto@LinuxServer: ~/Desktop". The user has run the command `php -v`. The output shows the PHP version `8.3.6 (cli)` (built: Mar 19 2025 10:08:38) (NTS). The output also shows the copyright information for The PHP Group, Zend Engine v4.3.6, Copyright (c) Zend Technologies, and with Zend OPcache v8.3.6, Copyright (c), by Zend Technologies.

```
cethfisto@LinuxServer:~/Desktop$ php -v
PHP 8.3.6 (cli) (built: Mar 19 2025 10:08:38) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.6, Copyright (c) Zend Technologies
with Zend OPcache v8.3.6, Copyright (c), by Zend Technologies
cethfisto@LinuxServer:~/Desktop$
```

Ahora crearemos el script de conexión en el directorio `/var/www/html`, usando el comando `sudo nano` llamaremos el archivo `conexion.php`.



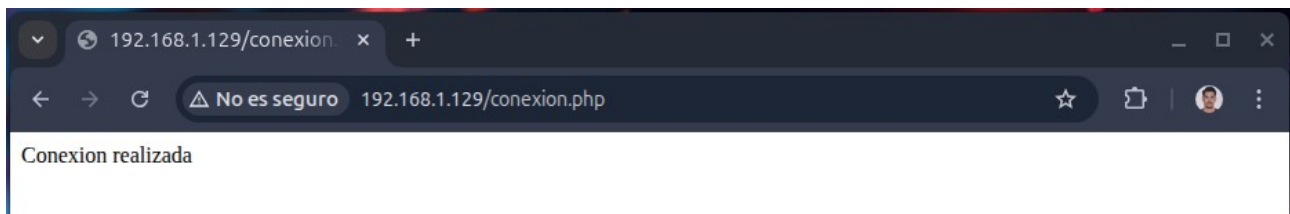
The screenshot shows a terminal window titled "Terminal - cethfisto@LinuxServer: ~/Desktop". The user has run the command `sudo nano /var/www/html/conexion.php`. The output shows the content of the `conexion.php` script. The script defines variables for host, database name, user, password, and charset. It then attempts to connect to the database using PDO and echoes the result.

```
GNU nano 7.2 /var/www/html/conexion.php *
<?php
$host = 'localhost';
$db = 'codearts_db';
$user = 'admin_db';
$password = '1234';
$charset = 'utf8mb4';

try {
    $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
    $pdo = new PDO($dsn, $user, $password);
    echo "Conexion realizada de forma correcta";
} catch (PDOException $e) {
    echo "Error de conexion: " . $e->getMessage();
}

echo "Conexion realizada";
?>
```

Verificaremos que podemos hacer la conexión usando la IP del servidor/`conexion.php`.



Ahora crearemos el script para hacer la pruebas de operaciones básicas, llamaremos al script test.php.

A screenshot of a terminal window titled "LinuxServer [Corriendo] - Oracle VirtualBox". The terminal shows the contents of a file named "test.php" edited in nano 7.2. The script is a PHP file that configures database connection variables, attempts a connection, creates a table if it doesn't exist, and inserts a record. The code is as follows:

```
<?php
// === CONFIGURACIÓN ===
$host = 'localhost';
$db = 'codearts_db';

// CAMBIA ESTE BLOQUE PARA PROBAR CON CADA USUARIO:
$user = 'intern_user'; // Puedes cambiar a admin_db o dev_user
$pass = '1234'; // Cambia la contraseña según el usuario

$charset = 'utf8mb4';
$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

echo "<h2>🔧 Probando conexión con el usuario: <code>$user</code></h2>";

try {
    $pdo = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
    ]);
    echo "✅ Conexión exitosa.<br><br>";
} catch (PDOException $e) {
    die("❌ Error de conexión: " . $e->getMessage());
}

// === CREAR TABLA SI NO EXISTE (si tiene permiso) ===
try {
    $pdo->exec("CREATE TABLE IF NOT EXISTS empleados (
        id INT AUTO INCREMENT PRIMARY KEY,
        nombre VARCHAR(100),
        puesto VARCHAR(100)
    )");
    echo "✅ Tabla verificada o creada.<br>";
} catch (PDOException $e) {
    echo "❌ No se pudo crear/verificar tabla: " . $e->getMessage() . "<br>";
}

// === INSERT ===
try {
    $pdo->exec("INSERT INTO empleados (nombre, puesto) VALUES ('Juan Pérez', 'Tester')");
    echo "✅ Insert exitoso.<br>";
} catch (PDOException $e) {
    echo "❌ Error en INSERT: " . $e->getMessage() . "<br>";
}
```

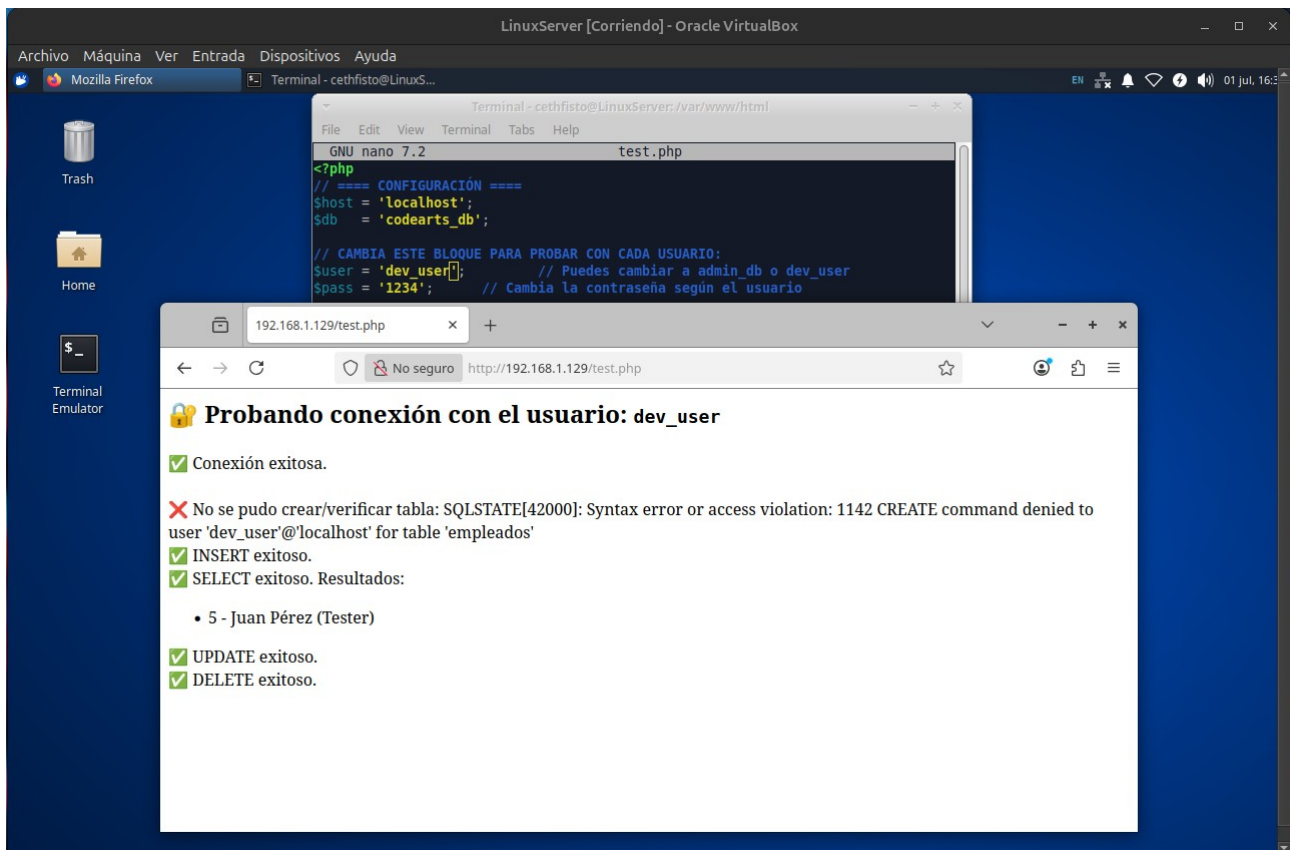
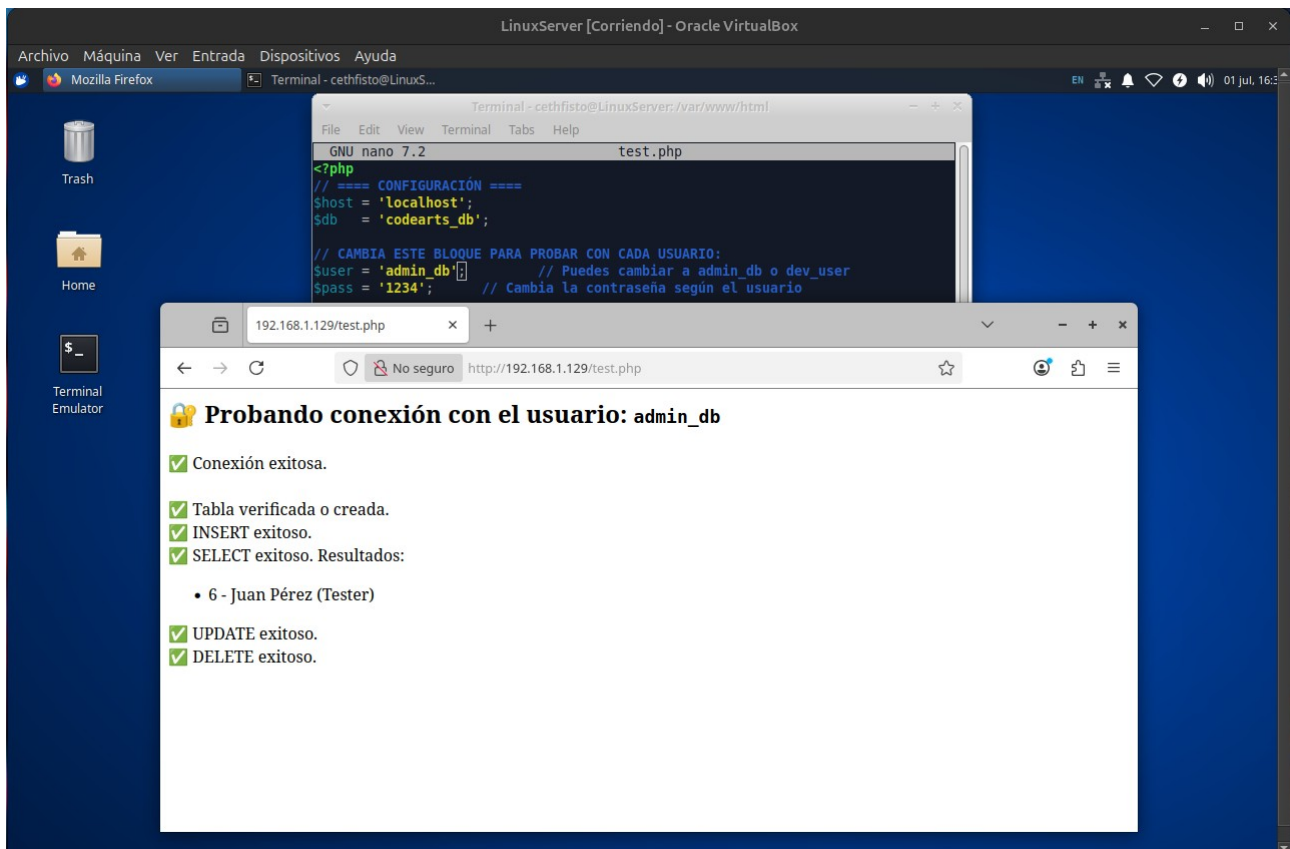
A continuation of the terminal window showing the rest of the "test.php" script. The code includes SELECT, UPDATE, and DELETE queries. The code is as follows:

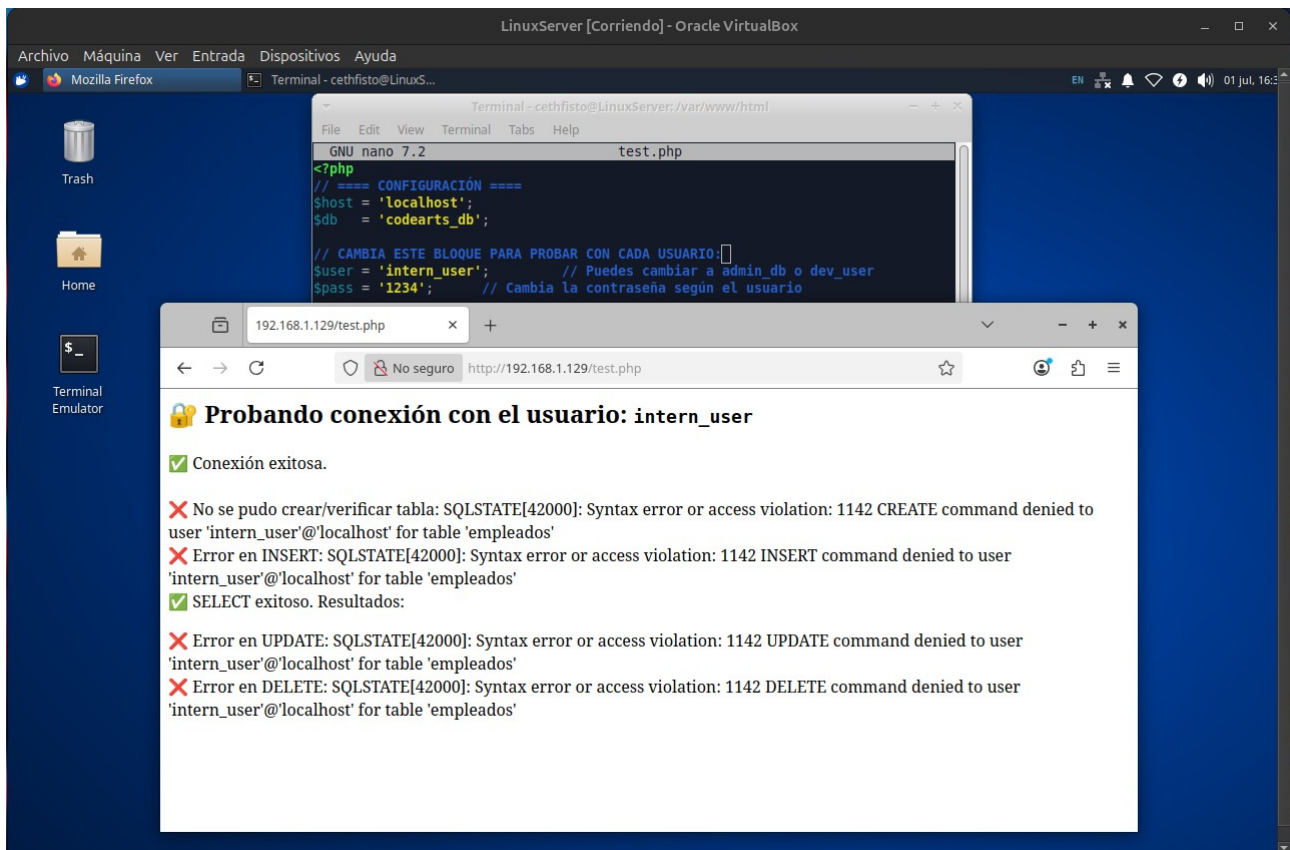
```
// === SELECT ===
try {
    $stmt = $pdo->query("SELECT * FROM empleados");
    echo "✅ SELECT exitoso. Resultados:<br><ul>";
    foreach ($stmt as $row) {
        echo "<li>{$row['id']} - {$row['nombre']} ({$row['puesto']})</li>";
    }
    echo "</ul>";
} catch (PDOException $e) {
    echo "❌ Error en SELECT: " . $e->getMessage() . "<br>";
}

// === UPDATE ===
try {
    $pdo->exec("UPDATE empleados SET puesto='Tester actualizado' WHERE nombre='Juan Pérez'");
    echo "✅ UPDATE exitoso.<br>";
} catch (PDOException $e) {
    echo "❌ Error en UPDATE: " . $e->getMessage() . "<br>";
}

// === DELETE ===
try {
    $pdo->exec("DELETE FROM empleados WHERE nombre='Juan Pérez'");
    echo "✅ DELETE exitoso.<br>";
} catch (PDOException $e) {
    echo "❌ Error en DELETE: " . $e->getMessage() . "<br>";
}
?>
```

Acto seguido nos conectaremos al servidor usando la IP del servidor/test.php, iremos cambiando de usuario para comprobar si tiene los permisos correctos en la tabla creada a la hora de gestionarla.



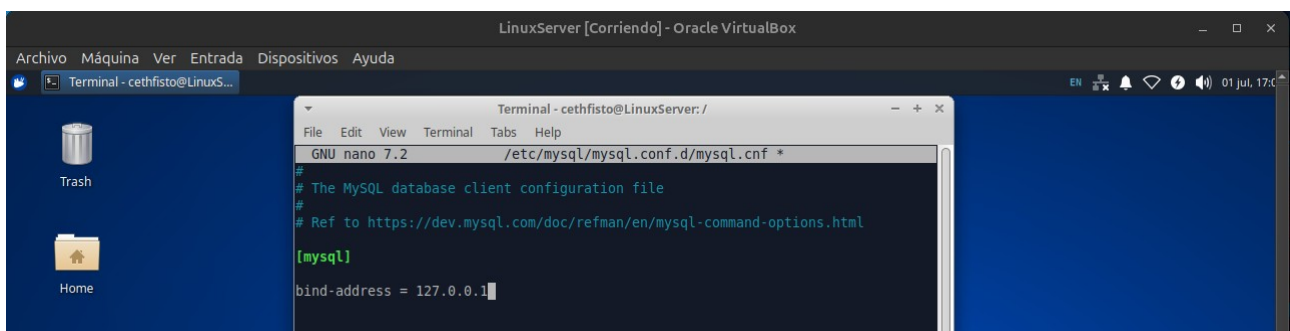


* Como podemos comprobar admin_db tiene todos los permisos, dev_user tiene los permisos de escritura y lectura y intern_user solo tiene permisos de lectura.

Fase: 4

En la penúltima fase crearemos accesos seguros limitando las conexiones remotas, implementaremos una autenticación con políticas de contraseñas, crearemos un sistema de backups automáticas y registraremos los intentos de acceso fallido y auditoria de consultas con logs del sistema.

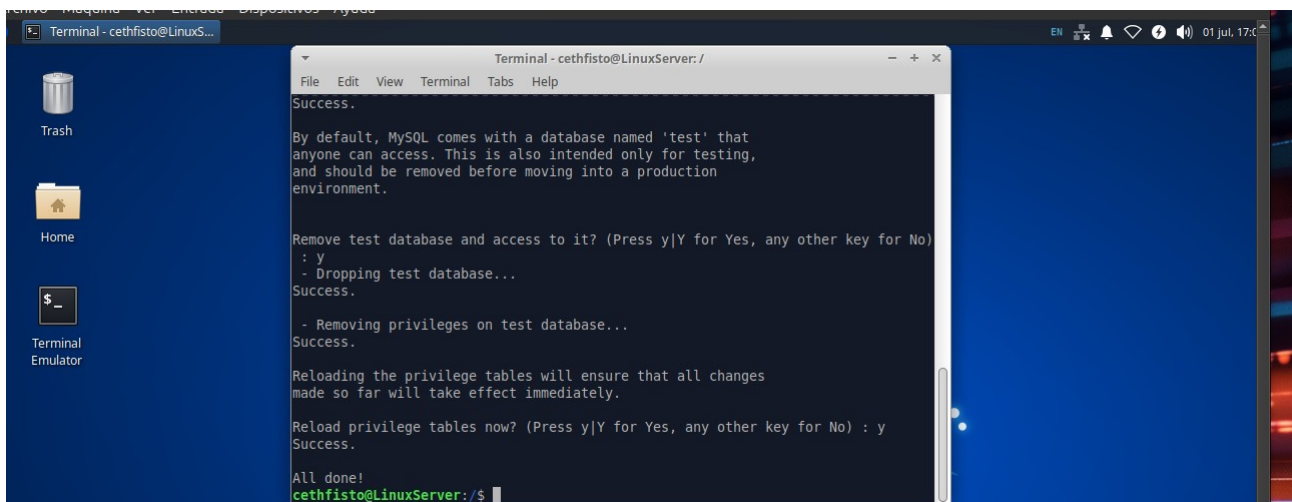
Normalmente MySQL escucha en 127.0.0.1, editaremos el archivo mysqld.cnf para confirmar que solo permite conexiones locales. Usaremos el comando `sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf` y reiniciamos el servicio con `sudo systemctl restart mysql`.



Para una autenticación segura y políticas de contraseñas, editamos el script integrado de MySQL usando el comando `sudo mysql_secure_installation`.

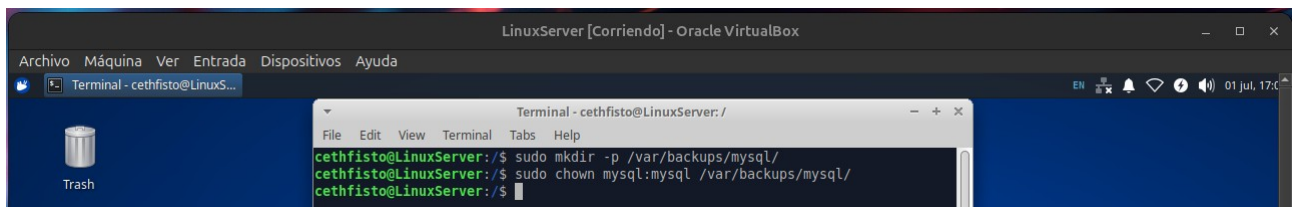
Los parámetros a configurar son:

- Validar la fortaleza de contraseñas : Si.
- Nivel de seguridad: 2 (fuerte)
- Cambiar contraseña de root : opcional
- Eliminar usuarios anónimos: Si.
- Deshabilitar acceso remoto de root: Si
- Eliminar BBDD de test: Si
- Recargar privilegios: Si.



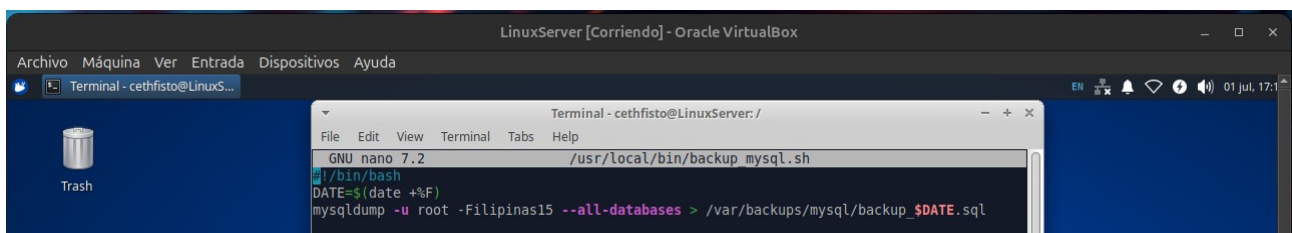
```
Terminal - cethfisto@LinuxServer: /
File Edit View Terminal Tabs Help
Success.
By default, MySQL comes with a database named 'test' that anyone can access. This is also intended only for testing, and should be removed before moving into a production environment.
Remove test database and access to it? (Press y|Y for Yes, any other key for No)
: y
- Dropping test database...
Success.
- Removing privileges on test database...
Success.
Reloading the privilege tables will ensure that all changes made so far will take effect immediately.
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.
All done!
cethfisto@LinuxServer:/$
```

A la hora de crear backups automáticos, crearemos un directorio para los backups y añadiremos el propietario en este caso MySQL.



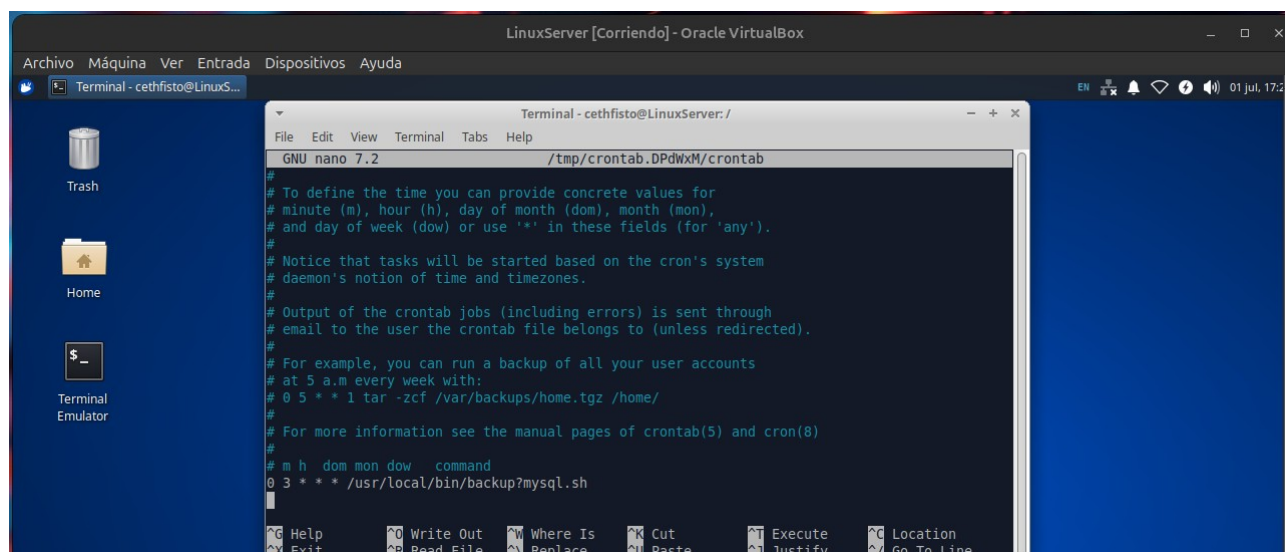
```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
Terminal - cethfisto@LinuxServer: /
File Edit View Terminal Tabs Help
cethfisto@LinuxServer:/$ sudo mkdir -p /var/backups/mysql/
cethfisto@LinuxServer:/$ sudo chown mysql:mysql /var/backups/mysql/
cethfisto@LinuxServer:/$
```

Crearemos un script de respaldo llamado backup_mysql.sh en /usr/local/bin/.



```
LinuxServer [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
Terminal - cethfisto@LinuxServer: /
File Edit View Terminal Tabs Help
GNU nano 7.2 /usr/local/bin/backup_mysql.sh
^I/bin/bash
DATE=$(date +%F)
mysqldump -u root -Filipinas15 --all-databases > /var/backups/mysql/backup_${DATE}.sql
```

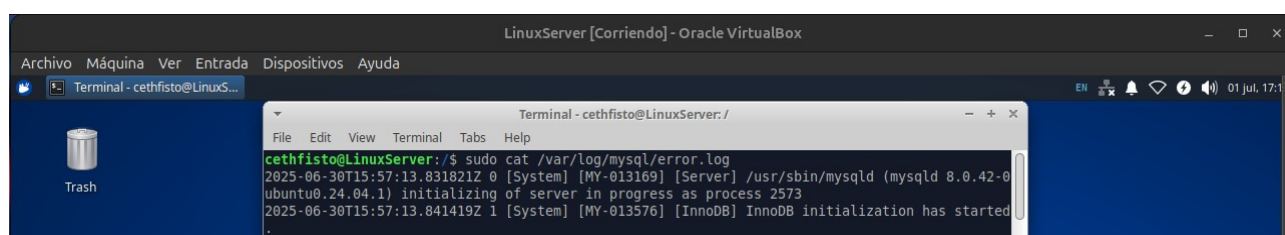
Daremos los permisos con el comando `sudo chmod +x /usr/local/bin/backup_mysql.sh` e iremos al archivo cron con el comando `sudo crontab -e` para añadir el comando de ejecución del backup.



```
GNU nano 7.2 /tmp/crontab.DPdWxM/crontab
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 3 * * * /usr/local/bin/backup_mysql.sh
```

* En este caso, se ejecutará el backup todos los días a las 3am.

Y para ver los logs de MySQL usaremos el comando `sudo cat /var/log/mysql/error.log`.



```
cethfisto@LinuxServer:/$ sudo cat /var/log/mysql/error.log
2025-06-30T15:57:13.831821Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.42-0
ubuntu0.24.04.1) initializing of server in progress as process 2573
2025-06-30T15:57:13.841419Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started
```

Fase: 5

En la última fase, argumentaremos cómo nuestro servidor de BBDD optimiza la gestión y seguridad de los datos en la empresa.

CodeArts ganará:

- Gestión mejorada y centralizada de datos
- Seguridad a la hora de proteger los datos y la información,
- Backups automatizados.
- Monitorización de los errores para tener más control.

Con esta configuración, el servidor de BBDD optimiza el rendimiento y la gestión de la info diaria. También protege los datos de la empresa frente amenazas externas e internas.